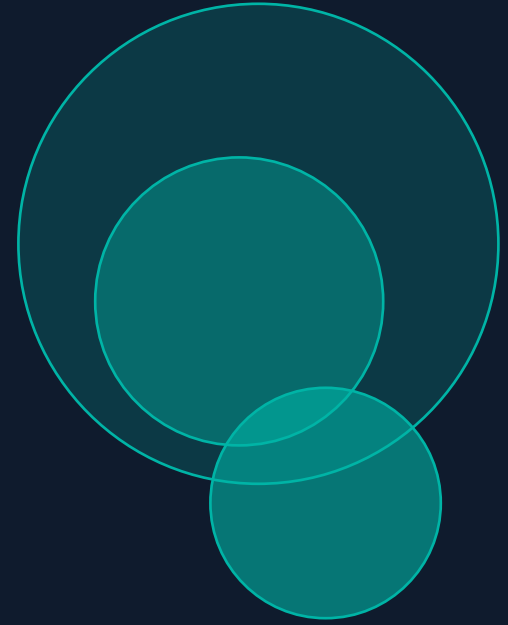


The Discount Dilemma

What Works, What Doesn't Work, What's Next

Are discounts really driving growth?



Group Dilemma Lab:

Mahsa Ahmadi,
Namratha Nagesh,
Abdul Rahman Abssi,
Ian Blair

THE HEADLINE FINDING

- **95.5%** of order lines are discounted
- **97.5%** of revenue comes from discounted products

Weak correlation **(0.26)** between discount depth and revenue

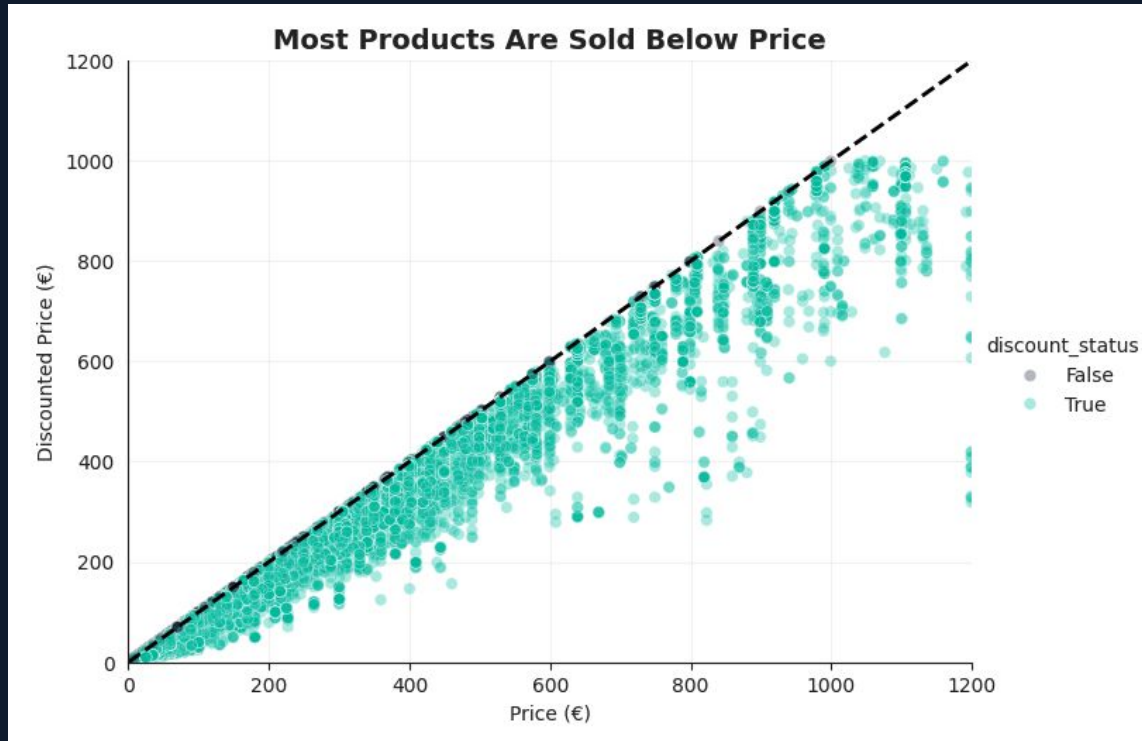
👉 Bigger discounts **do not** reliably increase revenue

Discounts are not a strategy — they are the default.

95.5%

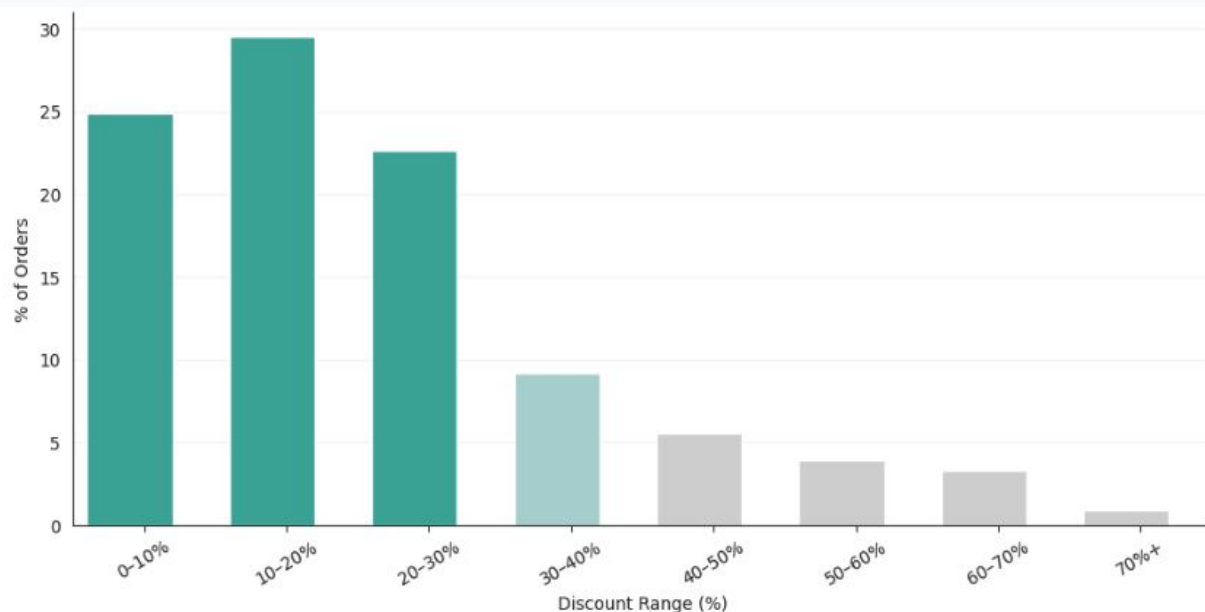
of all order lines are discounted

only **2.5%** full-price.



TYPICAL DISCOUNT DEPTH

Most discounts cluster between 10–25%. Deep cuts are rare — but the mean is being pulled up by a skewed tail.



19.4%

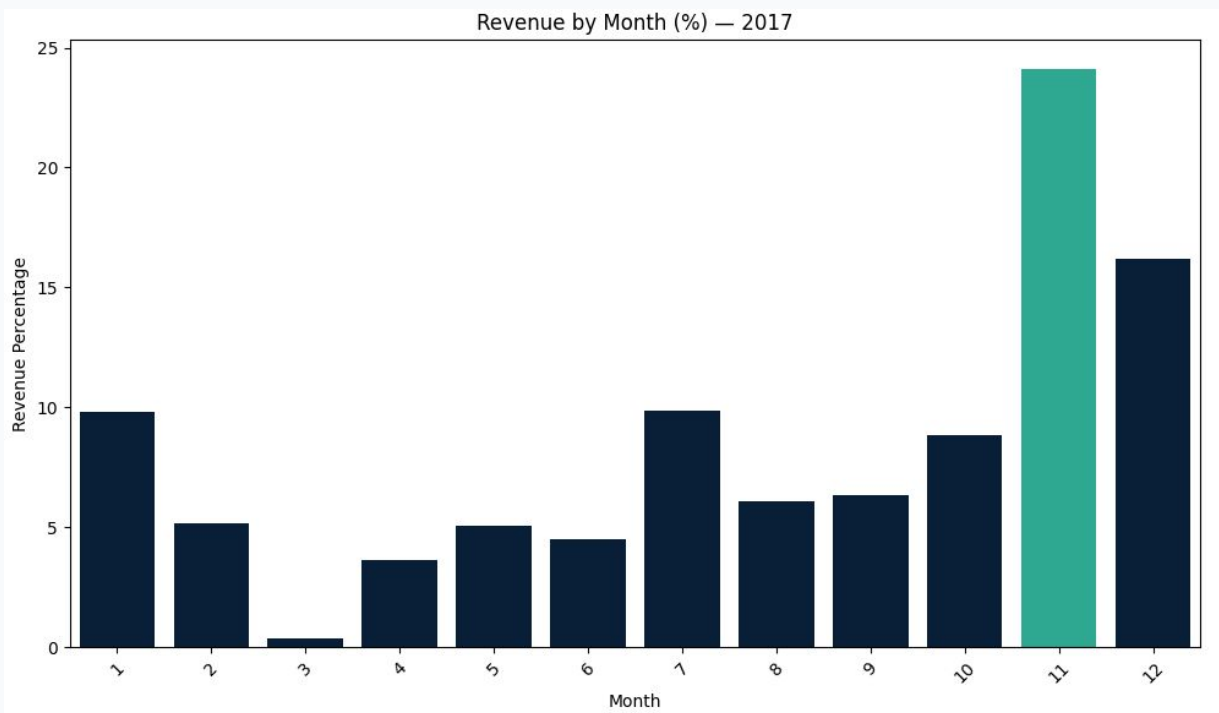
Median discount
(typical order)

~22.4%

Mean discount
(skewed by high-discount tail)

BLACK FRIDAY: THE PROOF IS IN THE VOLUME

November generates 3× average monthly revenue — at the same discount depth as any other month.



November
2017

6,573
orders

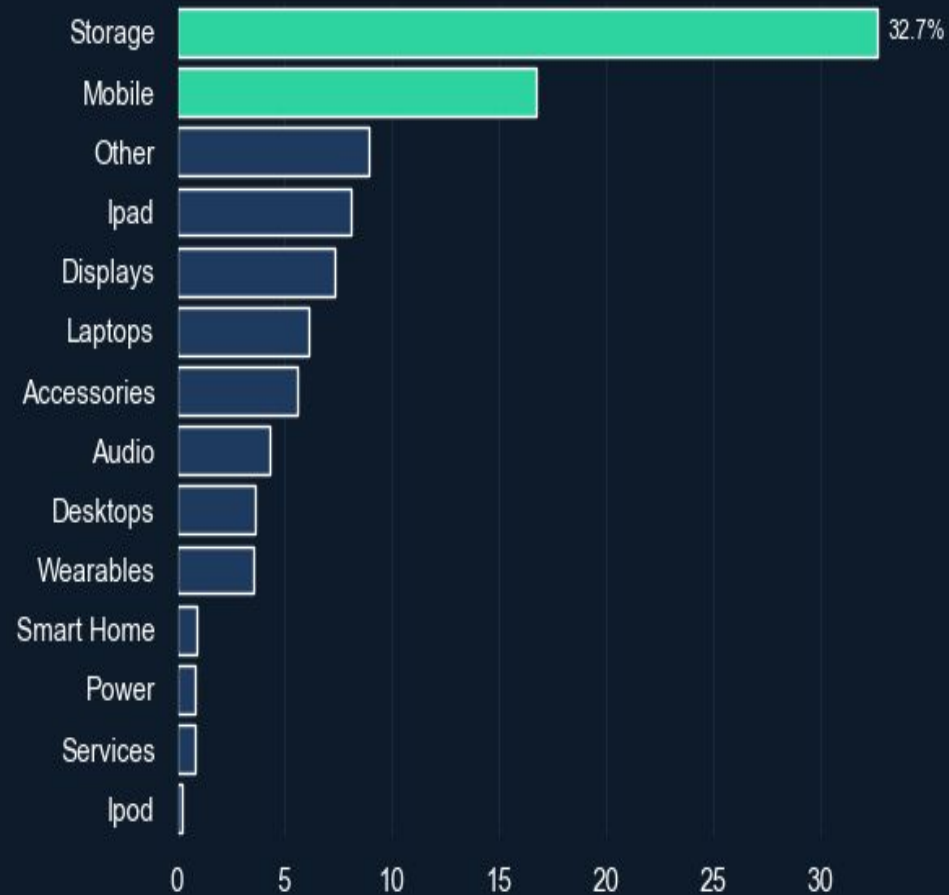
21.9%
avg discount

3×
monthly avg revenue

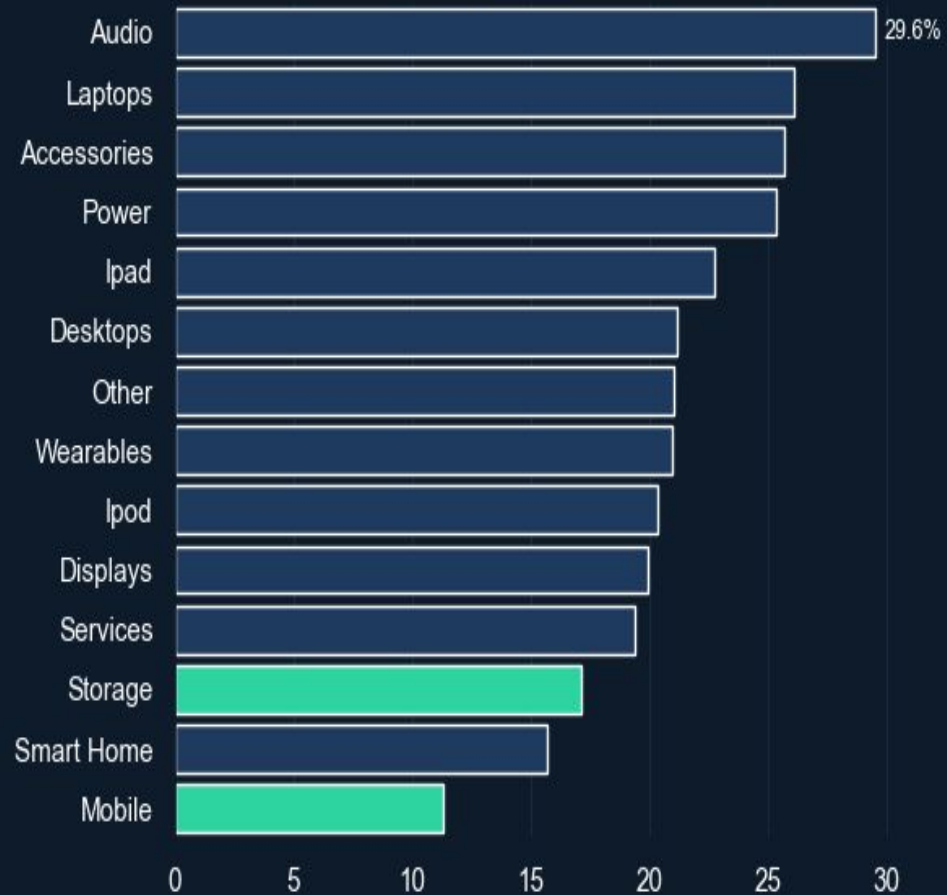
Volume is the lever.

Category Performance Overview

Revenue (%)



Avg Discount (%)



The question is not whether to discount. It's when, and by how much.

01

HIGH PRIORITY

90-Day No-Discount Test

Remove everyday discounts from the top 3 best-selling categories.

02

QUICK WIN

Protect Black Friday — But Cap Depth

November succeeds on volume, not discount depth.

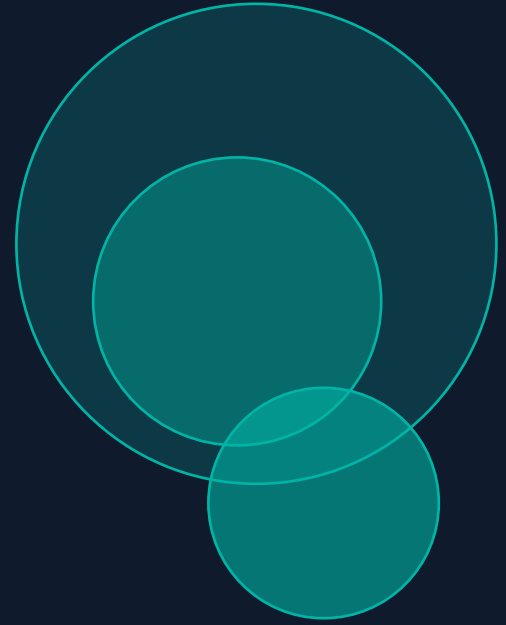
03

FOUNDATIONAL

Fix the Data Infrastructure

13.6% of order lines could not be categorised due to unreadable type codes.

The Discount Dilemma



Thanks for listening!

We will open it up for questions

Presented by: Namratha | Mahsa | Ian | Abdul

Appendix

Appendix 1: Monthly Revenue vs Average Discount %

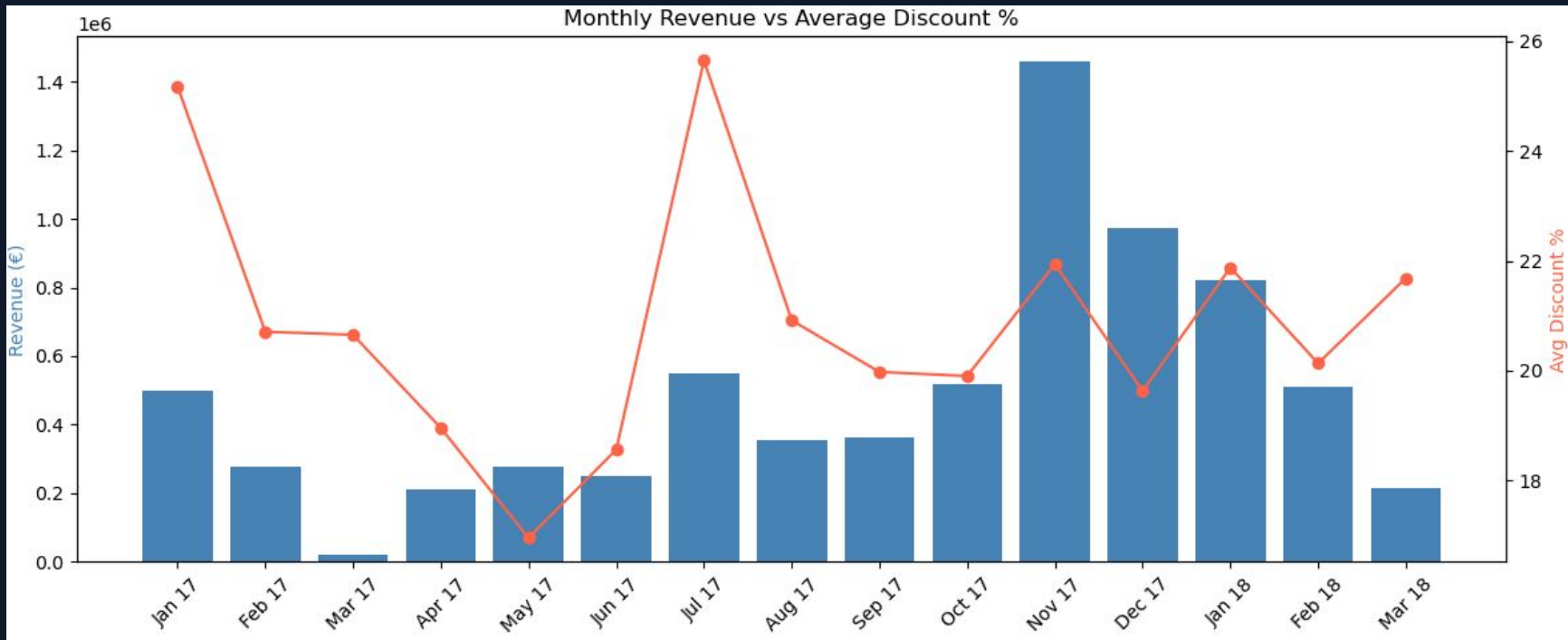
Appendix 2: Data Cleaning Steps

Appendix 3: Data Quality Issues and Recommended Fixes

Appendix 4: Product Categorization



Monthly Revenue vs Average Discount %



B1 · promo_price dropped

- 93.7% of rows had double-decimal errors (e.g. 1.029.99 instead of 1029.99)
- Could not be reliably parsed or imputed at scale
- Fix: Discount re-derived as `price - unit_price`
- Implication: planned vs actual discount comparison not possible with current data

B2 · Negative discounts removed

- Some rows had `unit_price > price`, implying a negative discount
- Caused by retroactive product price updates applied to old orders
- Fix: Rows where `discount < 0` were dropped
- Code: `df.loc[df['discount'] >= 0]`

B3 · Quantity × price anomalies

- Some rows: `unit_price × quantity > total_paid` (impossible)
- Indicates data inconsistency — line items exceed the order total
- Fix: Rows failing this integrity check were dropped
- Code: `df.loc[(df['unit_price'] * df['qty']) <= df['total_paid']]`

B4 · Date columns cast to datetime

- Both `created_date` and `date` stored as raw strings
- Needed datetime type for monthly resampling & time series
- Fix: `pd.to_datetime()` applied to both columns
- Enabled all seasonal and trend analysis in Section D

93.7%

promo_price rows corrupt

~3.4%

rows dropped

54,250

clean lines retained

4 steps

applied in sequence

Four structural data problems were identified that limit future analysis depth and should be addressed in the data infrastructure.

01 promo_price corruption

Problem: 93.7% of promo_price values had double-decimal formatting errors (e.g. 1.029.99). The column was entirely unusable and had to be dropped.

Fix: Validate prices at point of entry. Reject double-decimal strings before writing to the database. Add a DB constraint on the column format.

02 Unreadable type codes

Problem: The type column had 126 numeric codes with no lookup table. 13.6% of order lines fell into 'Other' because they could not be matched.

Fix: Add a product_categories lookup table mapping each type code to a human-readable name. Eliminates the keyword-matching workaround entirely.

03 No customer_id in exports

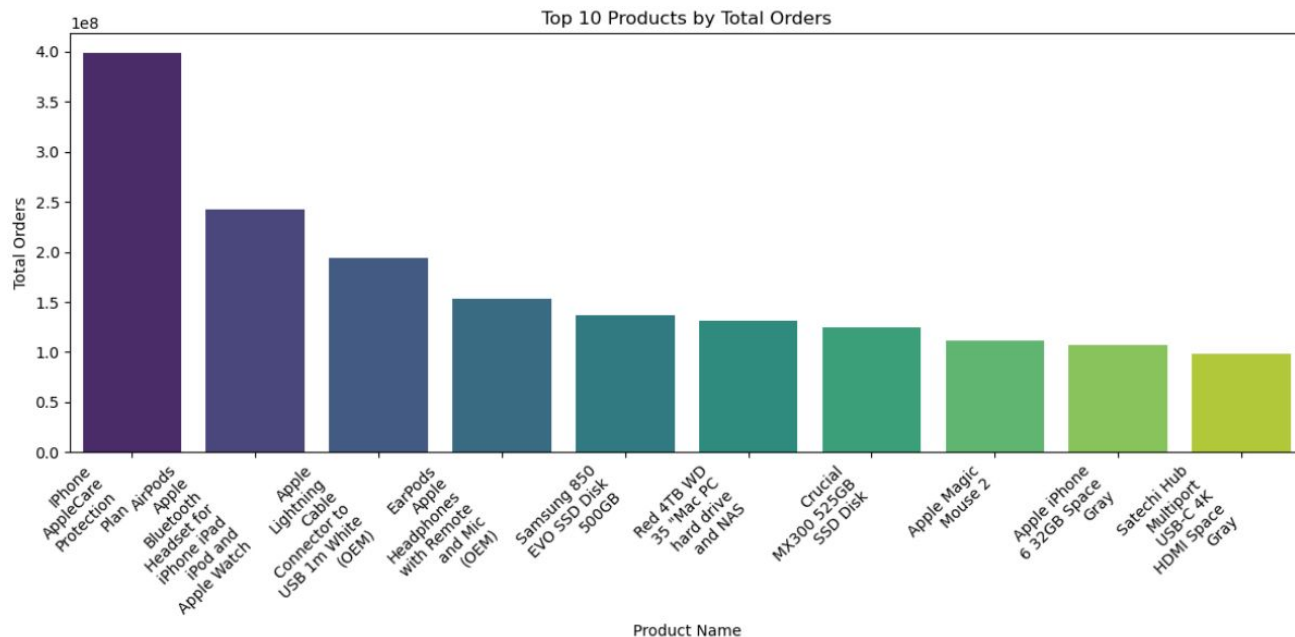
Problem: No customer identifier appears in the order export. Repeat-purchase rate, retention, and lifetime value are all unmeasurable.

Fix: Include an anonymised (HMAC-SHA256 hashed) customer_id in all order exports. GDPR-safe and enables cohort and retention analysis.

04 Retroactive price updates

Problem: Product prices are updated in the products table but old orders are not protected — causing 3.35% of rows to show unit_price > list price.

Fix: Snapshot the list price at the moment of each order. Add a price_at_order_time column to orderlines, populated on insert, never updated.

**Problem:**

Product categorisation initially caused major inconsistencies . This created gaps in analysis, reduced accuracy in reporting, and made it difficult to understand true product behaviour across segments..

Fix:

To fix this, we restructured the categorisation logic so that each product could accumulate multiple categories instead of being forced into just one.